Create a standalone Owl application

For any number of reasons, you may want to have a standalone Owl application that isn't a part of the web client. One example in Odoo is the self-ordering application, that lets customers order food from their phone. In this chapter we will go into what's required to achieve something like this.

Overview

To have a standalone Owl app, a few things are required:

- a root component for the application
- an assets bundle that contains the setup code
- a QWeb view that calls the assets bundle
- a controller that renders the view

1. Root component

To keep things simple, let's start with a very straightforward component that just renders "Hello, World!". This will let us know at a glance if our setup is working.

First, create the template in */your_module/static/src/standalone_app/root.xml* .

```
<?xml version="1.0" encoding="UTF-8"?>
<templates xml:space="preserve">
    <t t-name="your_module.Root">
        Hello, World!
    </t>
</templates>
```

Then create the JavaScript file for that component in */your_module/static/src/standalone_app/root.js* .

```
import { Component } from "@odoo/owl";

export class Root extends Component {
    static template = "your_module.Root";
```

odoo docs

It's generally a good idea to have the application setup code that mounts the component in a separate file. Create the JavaScript file that will mount the app in */your_module/static/src/standalone_app/app.js* .

```
import { whenReady } from "@odoo/owl";
import { mountComponent } from "@web/env";
import { Root } from "./root";

whenReady(() => mountComponent(Root, document.body));
```

The `mountComponent` utility function will take care of creating the Owl application and configuring it correctly: it will create an environment, start the **services**, make sure the app is translated and give the app access to the templates from your assets bundle, among other things.

> **See also**
>
> Owl components reference.

2. Creating an assets bundle containing our code

In the manifest of your module, create a new **assets bundle**. It should include the `web._assets_core` bundle, which contains the Odoo JavaScript framework and the core libraries it needs (e.g. Owl and luxon), after which you can have a glob that adds all the files for your application in the bundle.

```
{
    # ...
    'assets': {
        'your_module.assets_standalone_app': [
            ('include', 'web._assets_helpers'),
            'web/static/src/scss/pre_variables.scss',
            'web/static/lib/bootstrap/scss/_variables.scss',
            ('include', 'web._assets_bootstrap'),
            ('include', 'web._assets_core'),
            'your_module/static/src/standalone_app/**/*',
        ],
```

The other lines are bundles and scss files that are required to make Bootstrap work. They are mandatory, as the components of the web framework use bootstrap classes for their styling and layout.

> **Caution**
>
> Make sure that the files for your standalone app are only added to this bundle, if you already have a definition for `web.assets_backend` or `web.assets_frontend` and they have globs, make sure these globs don't match the files for your standalone app, otherwise the startup code for your app will conflict with the existing startup code in those bundles.

> **See also**
>
> Module manifest reference.

3. XML view that calls the assets bundle

Now that we have created our assets bundle, we need to create a QWeb view that uses that assets bundle.

```
<?xml version="1.0" encoding="utf-8"?>
<odoo>
    <template id="your_module.standalone_app">&lt;!DOCTYPE html&gt;
        <html>
            <head>
                <script type="text/javascript">
                    var odoo = {
                        csrf_token: "<t t-nocache="The csrf token must always be up
                        debug: "<t t-out="debug"/>",
                        __session_info__: <t t-esc="json.dumps(session_info)"/>,
                    };
                </script>
                <t t-call-assets="your_module.assets_standalone_app" />
            </head>
            <body/>
        </html>
```

This template only does two things: it initializes the `odoo` global variable, then calls the assets bundle we just defined. Initializing the `odoo` global variable is a necessary step. This variable should be an object that contains the following:

- The CSRF token, which is required to interact with HTTP controllers in many cases.
- The debug value, which is used in many places to add additional logging or developer-friendly checks.
- `__session_info__` , that contains information from the server that is always needed and for which we don't want to perform an additional request. More on this in the next section.

4. Controller that renders the view

Now that we have the view, we need to make it accessible to the user. For that purpose, we will create an **HTTP controller** that renders that view and returns it to the user.

```
from odoo.http import request, route, Controller

class YourController(Controller):
    @route("/your_module/standalone_app", auth="public")
    def standalone_app(self):
        return request.render(
            'your_module.standalone_app',
            {
                'session_info': request.env['ir.http'].get_frontend_session_info(),
            }
        )
```

Notice how we're giving the template `session_info` . We get it from the `get_frontend_session_info` method, and it will end up containing information used by the web framework, such as the current user's ID if they are logged in, the server version, the Odoo edition, etc.

At this point, if you open the url `/your_module/standalone_app` in your brower, you should see a blank page with the text "Hello, World!". At this point, you can start actually writing the code for your app.

odoo docs

Get Help

Contact Support | Ask the Odoo Community